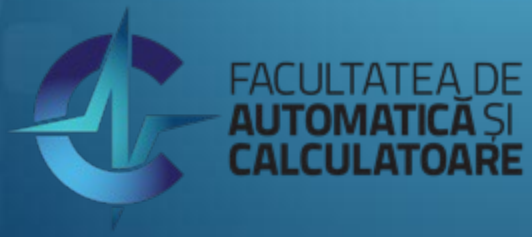


Algoritmi Paraleli și Distribuiți
Ceasuri Logice
Ordonarea evenimentelor

Radu-Ioan Ciobanu
radu.ciobanu@upb.ro





De la ceasuri fizice la... ceasuri logice

Ceasuri fizice

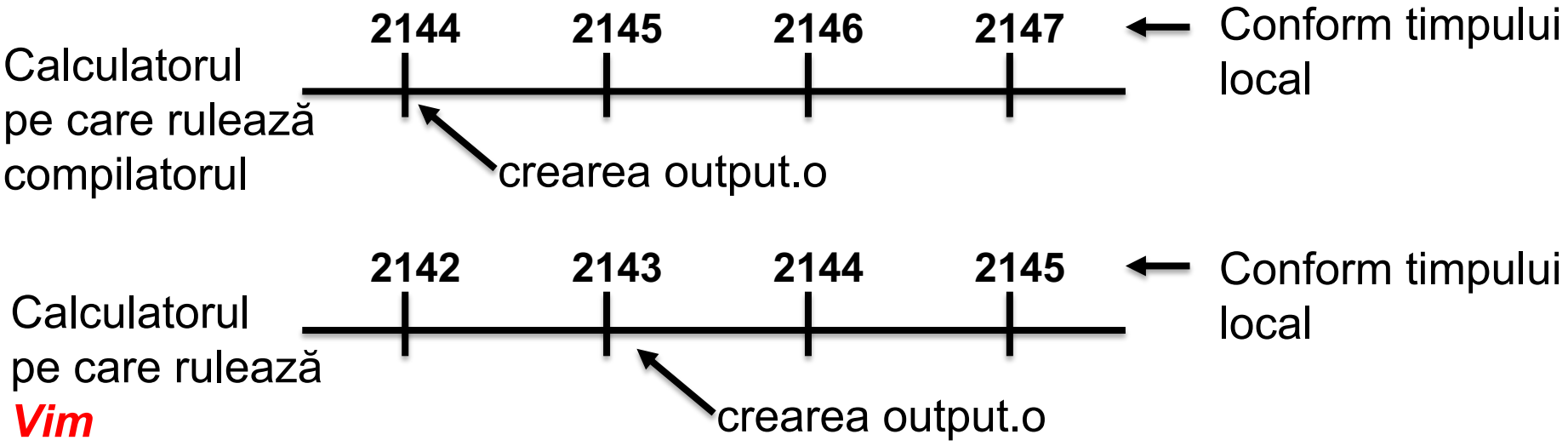
- Timpul este ne-ambiguu în sisteme ne-distribuite
  - există un singur ceas și (de obicei) kernelul face apeluri pentru a-i accesa valoarea
- Folosit implicit în multe aplicații
  - *Ex:* make *(pentru a determina fișierele care trebuie recompilate)*
- Sistemele distribuite introduc ambiguitatea în legătură cu timpul

Ceasuri fizice (2)

- Computerele folosesc un cristal de quartz care oscilează la o valoare bine definită și generează întreruperi (clock ticks) la intervale regulate
- Fiecare *clock tick* incrementează o valoare din memorie – kernelul convertește această valoare într-un format standardizat
  - *Ex:* nr. de milisecunde pornind de la Thu Jan 1 12:00:00 GMT 1970.
- Diferența în timp dintre calculatoare se numește clock skew.

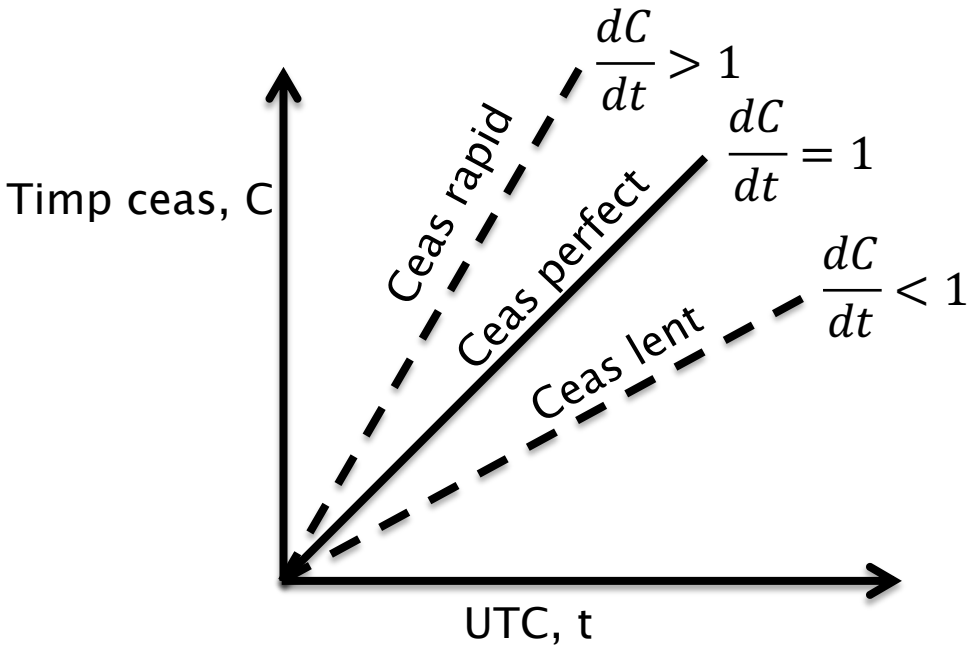
Ceasuri fizice (3)

- Clock skew dintre cele 2 computere confuzează make-ul:



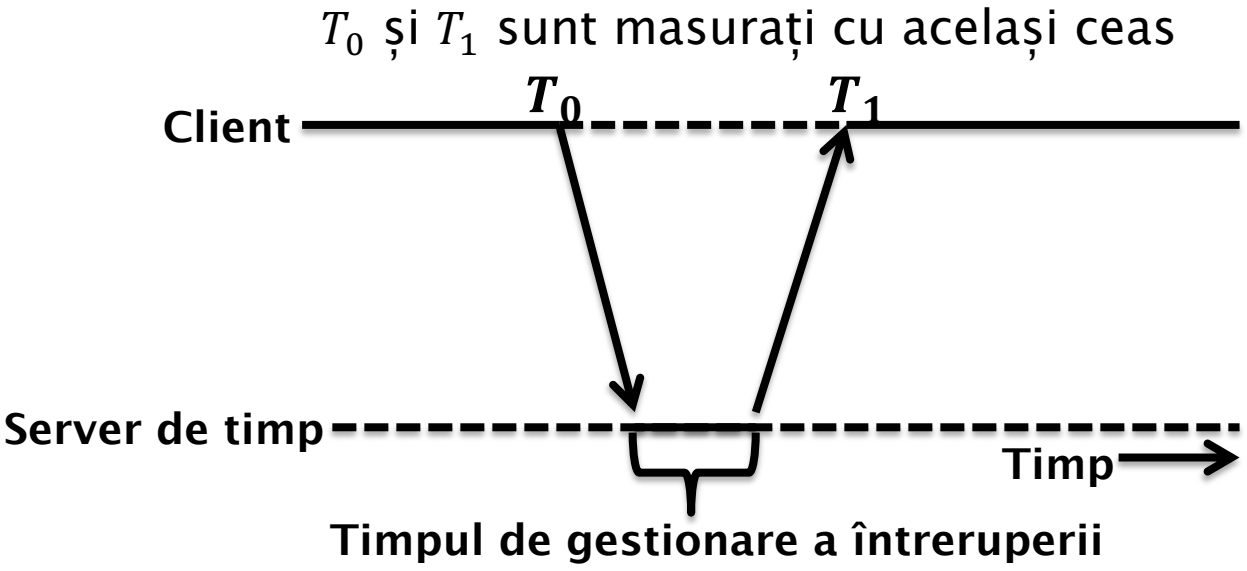
Sincronizarea ceasurilor fizice

- 2 tipuri:
  - doar între 2 calculatoare
  - sincronizare cu un standard *(ex: UTC)*
- Este imposibil de eliminat complet clock skew și imposibil de a asigura acuratețea perfectă a unui ceas.



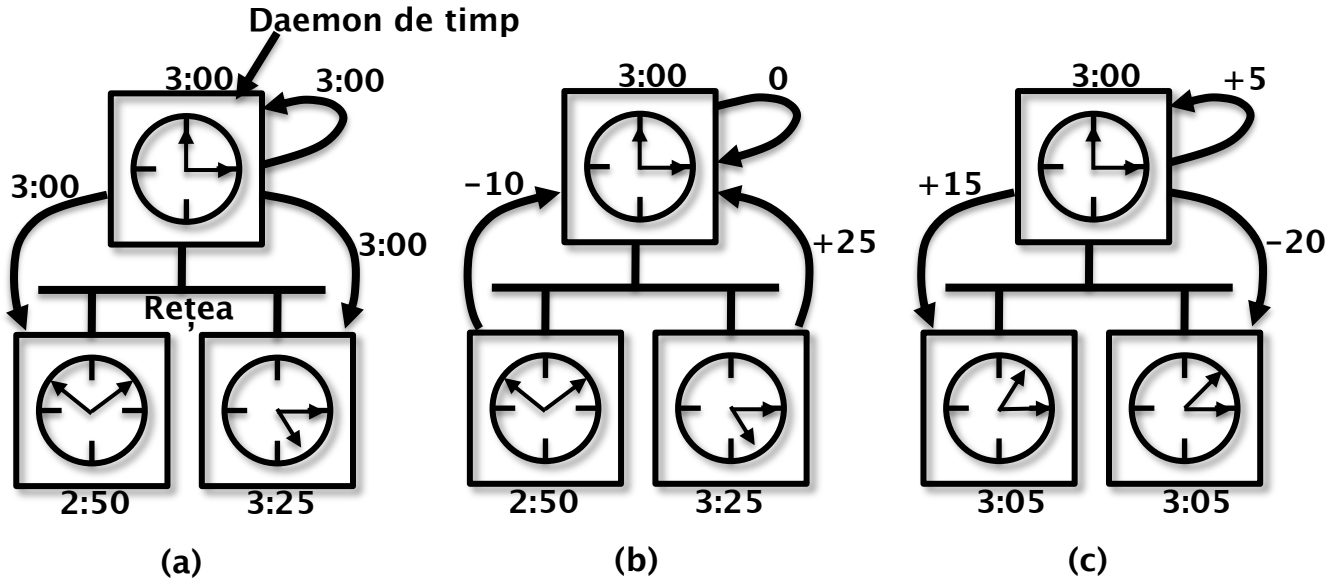
Sincronizarea ceasurilor fizice (2)

- Flaviu Cristian – adaptive internal clock synchronization:
  - time server, cu care se sincronizează celelalte ceasuri
  - probleme:
    - timpul nu trebuie sa curgă în sens invers
    - mesajul ajunge de la server la client într-un anumit timp



Sincronizarea ceasurilor fizice (3)

- Berkeley:
  - folosit în Berkeley UNIX
  - opus lui Flaviu Cristian*: un server de timp trimite mesaje periodic clienților pentru a afla timpul lor, calculează o medie și anunță clienții cum să-și actualizeze ceasurile*
  - scopul nu e de a se sincroniza cu time serverul, ci de a pune toate mașinile de acord asupra timpului (chiar dacă este greșit)



Sincronizarea ceasurilor fizice (4)

- Descentralizat: Network Time Protocol
  - Organizare ierarhică a serverelor de timp:
    - Stratul 1:
      - ceasuri de referință
        - *US Naval Observatory*
        - *Global Positioning System*
        - *Atomic Clocks*
    - Stratul 2:
      - routere, servere importante, etc.
  - Algoritmul și protocolul NTP extrem de complexe

    (RFC 1305 NTPv3 – 112 pagini , NTPv4 și mai complex)
  - Sincronizare precisă a ceasurilor: 1-50ms
  - https://www.ntppool.org/en/

Ceasuri logice (solutia Lamport)

Ceasuri logice și ordonarea evenimentelor

- Într-un algoritm distribuit, fiecare proces este caracterizat de:
  - o mulțime de stări
  - acțiuni care schimbă starea

- Eveniment *(producerea unei acțiuni)*
  - Evenimentele pot fi ordonate conform timpului fizic de producere.
  - Dificil pentru evenimente din procese diferite.

- Soluție mai simplă: ordinea relativă a evenimentelor

- Definită de relația petrecut înainte (→):
  - dacă **a** şi **b** sunt evenimente din acelaşi proces şi **a** îl precede in timp pe **b**, atunci a → b
  - când **a** reprezintă transmiterea unui mesaj de către un proces, iar **b** recepţia aceluiaşi mesaj de către un altul, atunci a → b
  - dacă a → b şi b → c, atunci a → c

Ceasuri logice și ordonarea evenimentelor (2)

- Mecanism de sincronizare bazat pe ***timp relativ***.
- ***timpul relativ*** poate să nu fie echivalent cu ***timpul real***.
- În aplicațiile distribuite ceea ce contează este ca procesele să *ajungă la un acord asupra ordinii de producere a* evenimentelor.
- Astfel de ***ceasuri*** se numesc ***ceasuri logice***.

Operating Systems

R. Stockton Gaines Editor

Time, Clocks, and the Ordering of Events in a Distributed System

Leslie Lamport
Massachusetts Computer Associates, Inc.

The concept of one event happening before another in a distributed system is examined, and is shown to define a partial ordering of the events. A distributed algorithm is given for synchronizing a system of logical clocks which can be used to totally order the events. The use of the total ordering is illustrated with a method for solving synchronization problems. The algorithm is then specialized for synchronizing physical clocks, and a bound is derived on how far out of synchrony the clocks can become.

Key Words and Phrases: distributed systems, computer networks, clock synchronization, multiprocess systems

CR Categories: 4.32, 5.29

Introduction

The concept of time is fundamental to our way of thinking. It is derived from the more basic concept of the order in which events occur. We say that something happened at 3:15 if it occurred *after* our clock read 3:15 and *before* it read 3:16. The concept of the temporal

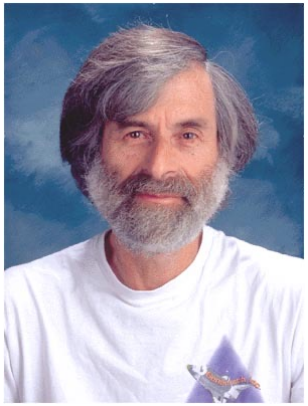
A distributed system consists of a collection of distinct processes which are spatially separated, and which communicate with one another by exchanging messages. A network of interconnected computers, such as the ARPA net, is a distributed system. A single computer can also be viewed as a distributed system in which the central control unit, the memory units, and the input-output channels are separate processes. A system is distributed if the message transmission delay is not negligible compared to the time between events in a single process.

We will concern ourselves primarily with systems of spatially separated computers. However, many of our remarks will apply more generally. In particular, a multiprocessing system on a single computer involves problems similar to those of a distributed system because of the unpredictable order in which certain events can occur.

In a distributed system, it is sometimes impossible to say that one of two events occurred first. The relation "happened before" is therefore only a partial ordering of the events in the system. We have found that problems often arise because people are not fully aware of this fact and its implications.

In this paper, we discuss the partial ordering defined by the "happened before" relation, and give a distributed algorithm for extending it to a consistent total ordering of all the events. This algorithm can provide a useful mechanism for implementing a distributed system. We illustrate its use with a simple method for solving synchronization problems. Unexpected, anomalous behavior can occur if the ordering obtained by this algorithm differs from that perceived by the user. This can be avoided by introducing real, physical clocks. We describe a simple method for synchronizing these clocks, and derive an upper bound on how far out of synchrony they can drift.

The Partial Ordering

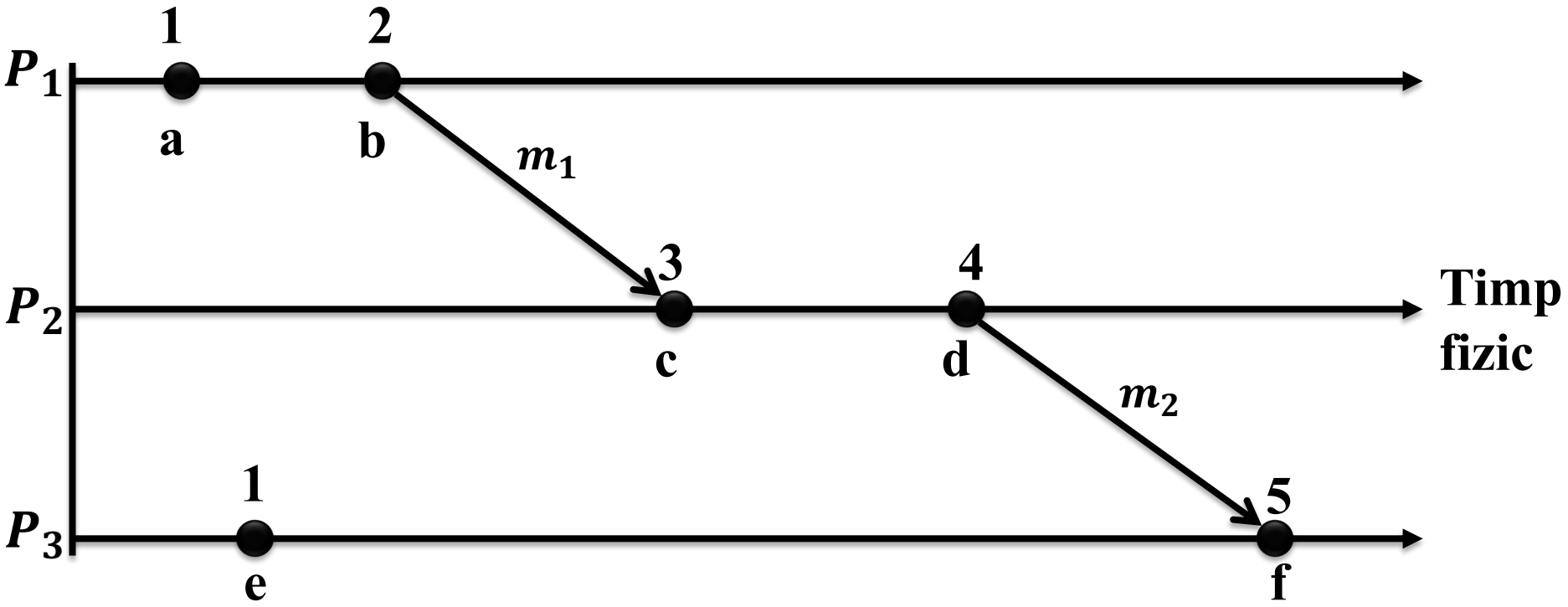


Lamport, Leslie. "Time, clocks, and the ordering of events in a distributed system." *Communications of the ACM* 21.7 (1978): 558-565.

Ceasuri logice și ordonarea evenimentelor (2)

- Ceas logic – întreg incrementat la producerea unui eveniment
- Algoritmul lui Lamport folosește relația → pentru a face ordonarea parțială a evenimentelor:
  - pentru fiecare proces o variabilă cl inițial 0
  - pentru fiecare mesaj un câmp tt
- Reguli:
  - la producerea unui eveniment intern:
    - cl este incrementat
    - valoarea ceasului logic este asociată evenimentului ca amprentă de timp
  - la transmiterea unui mesaj :
    - incrementează cl al procesului transmitator cu 1
    - actualizează tt = cl
  - la primirea unui mesaj cu amprenta de timp tt:
    - actualizează cl = maxim(cl, tt) + 1
- Ordonare parțială: $a \rightarrow b \Rightarrow cl(a) < cl(b)$

Exemplu



Aplicatie: semafoare distribuite

Semafoare distribuite

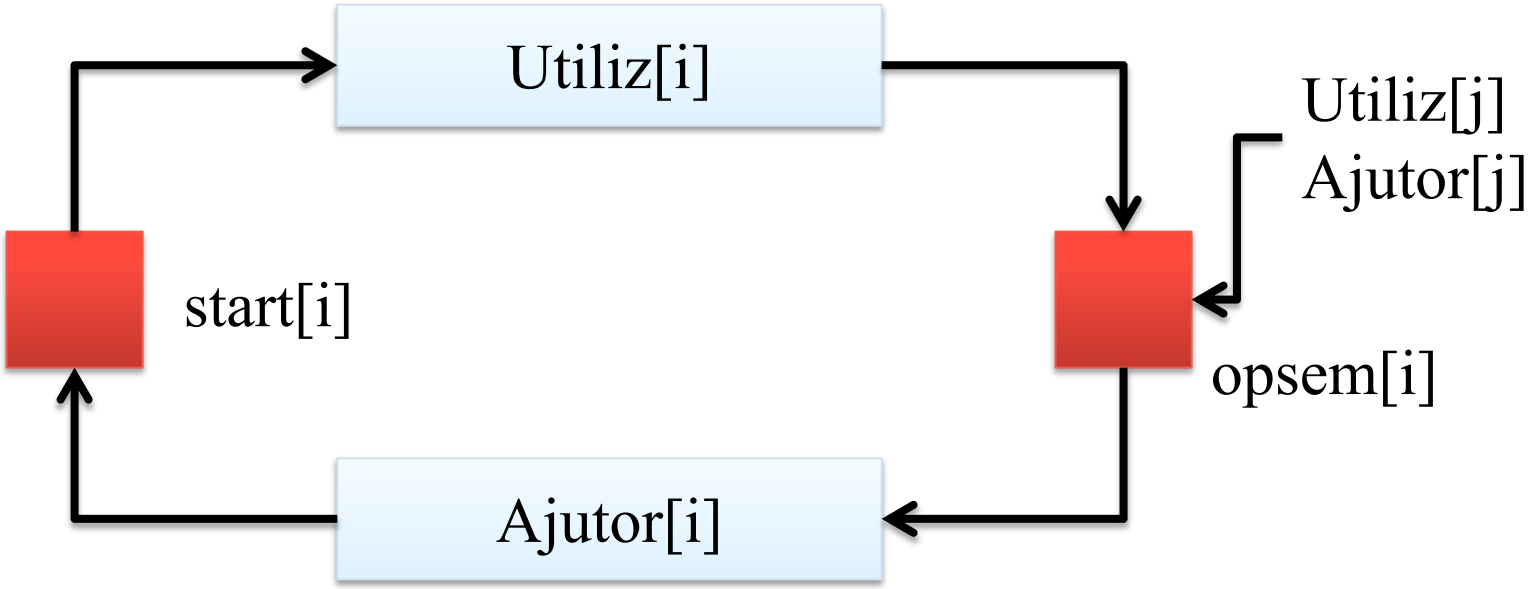
- Folosește primitive P și V
  - Când un proces execută o operaţie P sau V el difuzează mesaje m(id, tag, ts) celorlalte procese:

    broadcast ch(m) trimite m pe fiecare din ch[1:n]

    mesajul m are aceeasi amprenta de timp pe toate canalele
  - Analizează răspunsurile pentru a determina continuarea execuţiei
- Fiecare proces:
  - Stochează o coadă de mesaje mq și un ceas logic cl
  - La recepția unui mesaj P sau V îl stochează în mq, sortat crescător după ts
  - Transmite prin broadcast ACK *(folosit pentru a actualiza prefixele stabile din mq)*
  - Stochează o variabila s *(semaforul)*:
    - V – s este incrementat și mesajul șters
    - P – dacă $s > 0$, s decrementat și mesajul șters
      - mesajele P sunt procesate în ordinea în care apar în prefixul stabil deci fiecare proces ia aceeași decizie despre ordinea terminării operațiilor P

Aplicație: semafoare distribuite (2)

- Implementarea semafoarelor:
  - procesele Utiliz(i) inițiază operațiile P sau V
  - procesele Ajutor(i) implementează operațiile P și V



Aplicație: semafoare distribuite (3)

```
enum fel(V, P, ack);
chan opsem[1:n](int transm, fel op, int timp);
chan start[1:n](int timp);

process Utiliz[i= 1 to n] {
  int cl = 0;                          /* ceas logic */
  int ts;
  ...
  cl = cl + 1;
  broadcast opsem(i, V, cl);           /* operația V */
  ...
  cl = cl + 1;
  broadcast opsem(i, P, cl);           /* operația P */
  receive start[i](ts);
  cl = max(cl, ts)+1;
  ...
}
```

Aplicație: semafoare distribuite (4)

```
process Ajutor[i = 1 to n]{
    typedef struct{int transm, fel k, int ts) qelem;
    qelem qm[lmax]; /*coada ordonata dupa timestamp tm*/
    int cl = 0;
    int sem = valoare_inițială;
    int transm; fel k; int ts;
    while (true) {
        receive opsem[i](transm, k, ts);
        cl = max(cl, ts) + 1;
        if (k == P or k == V) {
            inserează (transm, k, ts) în locul corespunzător în qm;
            cl = cl + 1; broadcast opsem(i, ack, cl); }
        else if (k == ack) {
            înregistrează transmiterea unui ack;
            for [mesajele V complet confirmate] {
                scoate mesaj din qm;
                sem = sem + 1; }
            for [mesajele P complet confirmate st sem > 0] {
                scoate mesaj (transm, k, ts) din qm; // atentie la coliziuni
                sem = sem - 1;
                if (transm == i) {cl = cl + 1; send start[i](cl);} }
        }
    }
}
```

Ceasuri logice vectoriale

Ceasuri logice vectoriale

Cu soluţia Lamport:

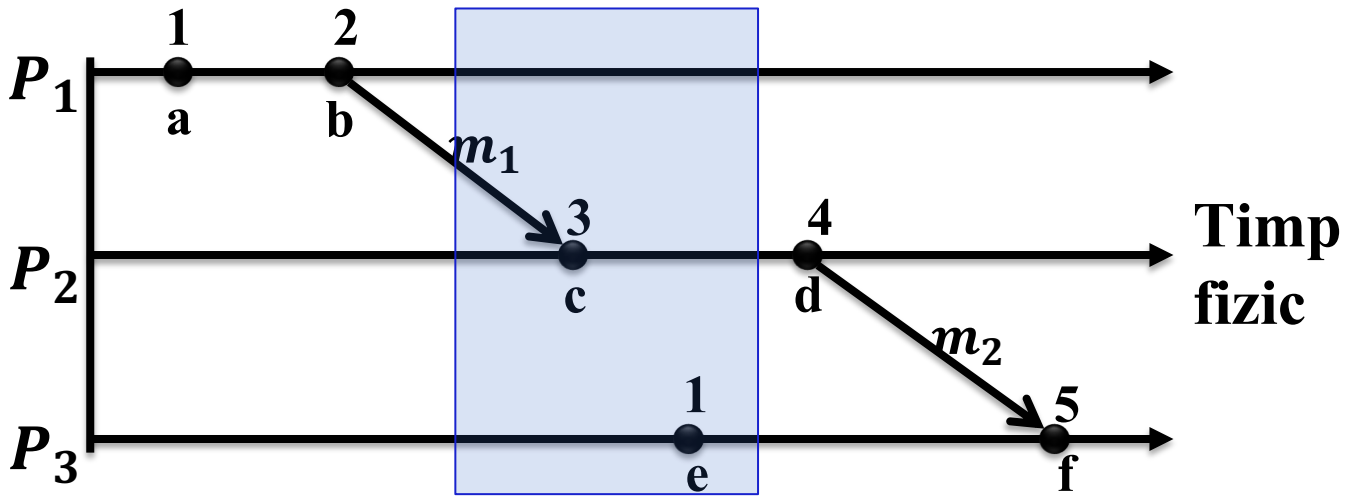
e precede f ⇒ amprenta_logică (e) < amprenta_logică (f)

Dar

amprenta_logică (e) < amprenta_logică (f) ⇏ e precede f

Ex: se poate spune ca **e** precede **c** ?

Soluția: ceasuri logice vectoriale.



Ceasuri logice vectoriale (2)

- Fiecare P_i are asociat un tablou $V_i[1..n]$ în care:
  - $V_i[i]$ este numărul de evenimente produse în procesul P_i
  - $V_i[j]$ este numărul de evenimente despre care P_i ştie *(a aflat)* că au avut loc la P_j.

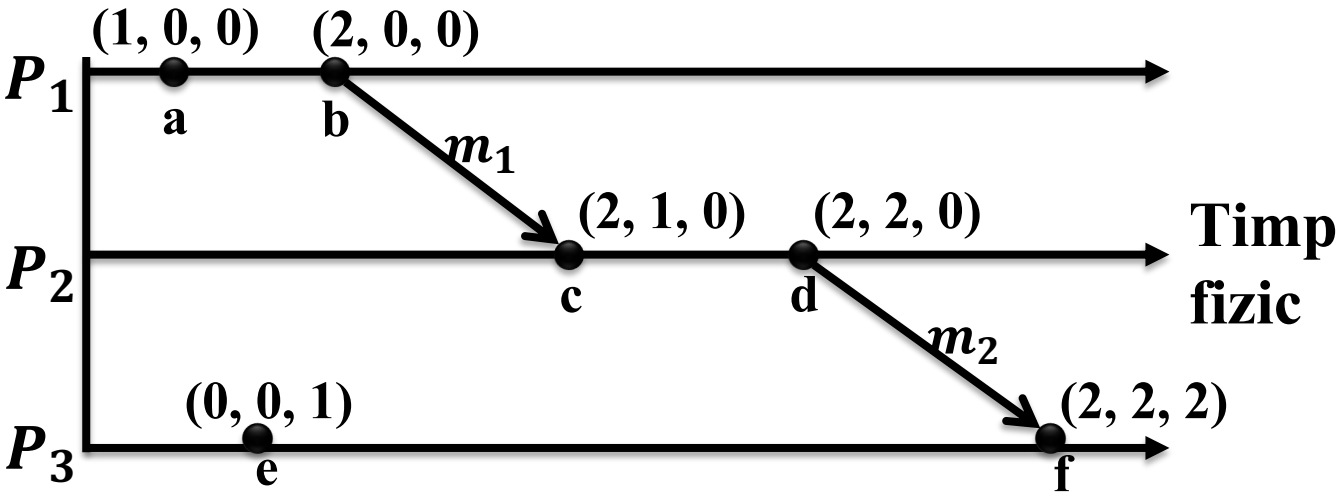
- Procesul P_i actualizează V_i la fiecare **eveniment** din P_i
  - e.g. pentru procesorul 3, (1,2,1,3) → (1,2,2,3)

- Când P_i **transmite** mesajul *m* *(eveniment send m):*
  - P_i incrementează $V_i[i]$
  - P_i adaugă V_i la *m* ca vector de amprente de timp curent $\boldsymbol{vt_m}$

- Când P_j **primeşte** *m* şi $\boldsymbol{vt_m}$ *(eveniment receive m):*
  - P_j ajustează: $V_j[k] = max\{V_j[k], vt_m[k]\}$ pentru fiecare k

    e.g., P_2 primește un mesaj cu timpul (3,2,4) iar timpul curent al lui P_2 este (3,4,3), atunci P_2 ajusteaza timpul la (3,4,4)
  - P_i incrementează $V_j[j]$ cu 1

Ceasuri logice vectoriale (3)



Aplicarea regulilor ceasurilor logice vectoriale

Reguli:

- $VT_1 = VT_2 \Leftrightarrow VT_1[i] = VT_2[i]$, pentru $i = 1, \dots, N$
- $VT_1 \leq VT_2 \Leftrightarrow VT_1[i] \leq VT_2[i]$, pentru $i = 1, \dots, N$
- $VT_1 < VT_2 \Leftrightarrow VT_1[i] \leq VT_2[i]$, și $VT_1 <> VT_2$ *(de exemplu (1,2,2) < (1,3,2))*

Fie vt(a) şi vt(b) vectorii de amprente de timp asociaţi ev. a şi b. Atunci:

vt(a) < vt(b) $\Rightarrow$ **evenimentul a precede cauzal b**

vt(a) $\not<$ **vt(b) and vt(a)** $\not>$ **vt(b) and vt(a)** $\neq$ **vt(b)** $\Leftrightarrow$ **evenimentele a şi b sunt concurente**

Aplicație: Ordonare Cauzală Multicast

Ordonare Cauzală Multicast

- Procesele unei colecţii P comunică între ele doar prin mesaje cu difuzare
- Se cere ca mesajele să respecte **dependenţa cauzală**

$$m \rightarrow m' \Rightarrow livrarep(m) \rightarrow livrarep(m')$$

Protocolul *(vectori de timp)*:

- fiecare proces $P_i = (i = 1..n)$ are asociat un vector $V_i[1..n]$, cu toate elementele iniţial 0
- se numara doar operatiile de transmitere de mesaje
- $V_i[i]$ este nr ev. transmitere de mesaje produse de P_i;
- $V_i[j]$ este nr ev. transmitere de mesaje despre care $\boldsymbol{P_i}$ ştie (a aflat) că au avut loc la P_j.
- Procesul P_i actualizează V_i la fiecare eveniment de trimitere sau recepţie de mesaj din P_i.

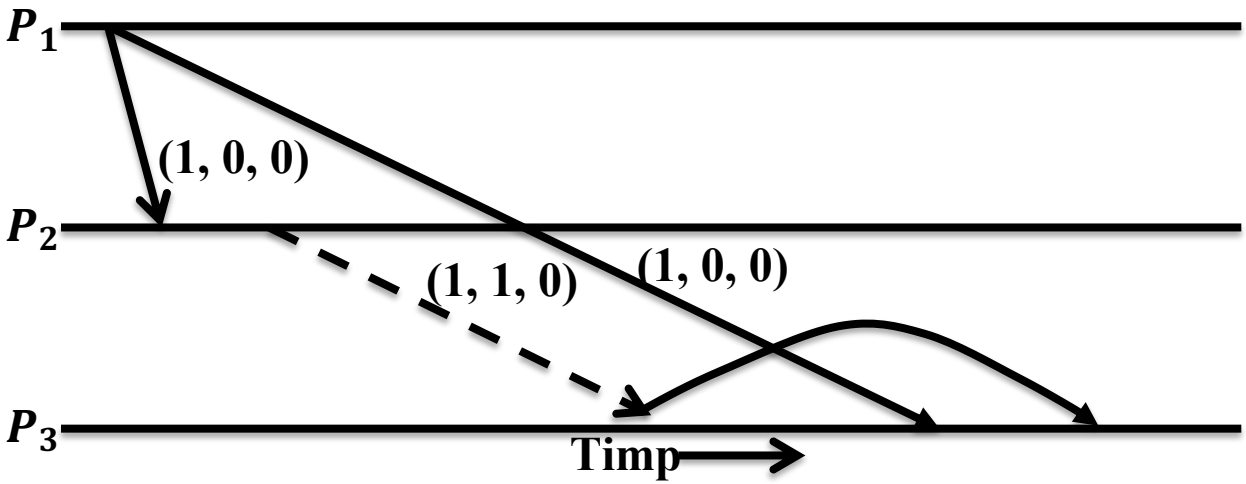
Ordonare Cauzală Multicast (2)

- Când P_S **transmite** mesajul *m*:
  - $\boldsymbol{P_S}$ **incrementează** $\boldsymbol{V_S[s]}$
  - $\boldsymbol{P_S}$ **adaugă** $\boldsymbol{V_s}$ **la** ***m*** **ca vector de amprente de timp curent** $\boldsymbol{vt_m}$
    - **Obs: Pentru ordonare cauzală, incrementarea lui** $\boldsymbol{V_s[s]}$ **se face doar la transmiterea de mesaje de către s**
    - vt_m spune receptorului câte evenimente (din alte procese) au precedat **m** şi ar putea influenţa cauzal pe **m**.

- Când P_d **primeşte** mesajul *m* împreună cu vt_m, mesajul este păstrat într-o coadă de **întârziere** şi este **livrat** doar dacă:
  - $\boldsymbol{vt_m[s] = V_d[s] + 1}$ (acesta este următorul timestamp pe care **d** îl așteaptă de la **s**)
  - $\boldsymbol{vt_m[k] \leq V_d[k]\ pentru\ k <> s}$ (**d** a văzut toate mesajele ce au fost văzute de **s** la momentul când a trimis mesajul **m**)

- Când mesajul este livrat, V_d este actualizat conform regulilor vectorilor de timp:
  $\boldsymbol{V_d[k] = max\{V_d[k],\ vt_m[k]\}}$ **pentru fiecare** $\boldsymbol{k = 1, n}$**.**

Ordonare Cauzală Multicast (3)



Alte acţiuni ale protocolului la livrarea mesajelor:

- dacă $vt_m[k] > V_d[k]$ pentru un oarecare k atunci se întârzie *m*

P_S a primit mesaje de care mesajul curent poate fi cauzal dependent, dar pe care P_d încă nu le-a primit;

- dacă $vt_m[s] > V_d[s] + 1$ atunci se întârzie *m*

mai sunt mesaje de la P_S pe care P_d nu le-a primit

(asigură ordinea FIFO pentru canale nonFIFO)

- dacă $vt_m[s] < V_d[s]$ atunci rejectează *m*

m este un duplicat al unui mesaj primit anterior

Sumar

- Ceasuri fizice
- Ceasuri logice și ordonarea evenimentelor
- Semafoare distribuite
- Ceasuri logice vectoriale
- Ordonare cauzală multicast